

Phase 1: Spatial Preprocessing (PiP Extraction)

This phase focuses on extracting the **Sign Language Interpreter (SLI)** region from broadcast videos where the interpreter appears as a **Picture-in-Picture (PiP)** window. The goal is to isolate the interpreter from complex parliamentary backgrounds and produce consistent, high-quality visual inputs for downstream processing.

The phase implements a **complete preprocessing pipeline** that includes video initialization, multi-method interpreter detection, parameter tuning, cropping, segmentation, quality control, and dataset organization.

1.1 Input Initialization

The process begins by loading the input broadcast video and extracting basic metadata required for processing.

Technologies Used:

- OpenCV (VideoCapture)
- NumPy

Functionality:

- Load video file
- Extract frame dimensions (width, height)
- Retrieve frame rate (FPS)
- Determine total frame count and duration

This information is used to guide all subsequent processing steps.

1.2 Multi-Method Interpreter Detection System

The system uses an **Auto Detection Mode**, which applies multiple detection strategies in order of reliability to locate the PiP interpreter region.

• Border Detection (Primary Method)

Detects light-colored PiP borders commonly used in broadcast layouts.

Techniques:

- HSV color space conversion
- Color thresholding
- Morphological operations (opening and closing)
- Contour detection

Process:

- Identify light regions using HSV thresholds
- Clean noise using morphological filters
- Detect rectangular contours
- Extract inner PiP region using configurable border margin (5%–15%)

Strength: Highly accurate for standard broadcast PiP layouts

• Motion Analysis (Fallback 1)

Identifies interpreter regions based on motion intensity.

Techniques:

- Optical Flow (Farneback algorithm)
- Motion magnitude computation
- Temporal motion accumulation

Process:

- Compute motion between consecutive frames
- Accumulate motion heatmap over time
- Identify high-motion regions (typically PiP corners)

Strength: Effective when borders are not visible but signing motion is strong

• Edge Detection (Fallback 2)

Detects PiP regions using structural edge information.

Techniques:

- Canny edge detection
- Contour extraction
- Polygon approximation

Process:

- Extract edges from grayscale frames
- Accumulate edge density across frames
- Detect rectangular shapes in corner regions
- Validate aspect ratio consistency

Strength: Works well for static overlays with clear boundaries

- **Pose Detection (Optional Method)**

Uses human pose estimation to locate the interpreter.

Technology:

- MediaPipe Pose (33 body landmarks)

Process:

- Detect body keypoints (shoulders, arms, wrists)
- Estimate bounding box from landmarks
- Validate visibility and corner positioning
- Confirm interpreter presence across frames

Strength: Useful when visual PiP cues are weak

1.3 Parameter Application

After detection, user-defined parameters refine the extracted region.

Parameters:

- **Border Margin (0.0–0.5):** Adjusts cropping tightness
- **Crop Adjustment ($\pm$ pixels):** Expands or shrinks bounding box
- **Start Time Offset:** Skips irrelevant initial content

Processing:

- Adjust detected coordinates
- Clamp values within frame boundaries
- Ensure safe cropping region

1.4 Video Cropping

The detected region is extracted using FFmpeg-based cropping.

Technology:

- FFmpeg (subprocess execution)

Process:

- Apply crop filter using detected coordinates
- Preserve original audio stream
- Optimize encoding speed using fast preset

Output:

- Cropped video containing only interpreter region

1.5 Clip Segmentation

The cropped video is divided into fixed-duration clips for dataset creation.

Configuration:

- Clip duration: 5 seconds
- Optional overlap: 0%

Process:

- Compute total number of clips
- Extract sequential segments using FFmpeg
- Maintain synchronized audio in each clip
- Assign structured filenames (e.g., clip_0001.mp4)

1.6 Resizing (Optional Standardization)

All clips can be resized to a uniform resolution for model training.

Technology:

- OpenCV (cv2.resize)

Standard Sizes:

- 128×128
- 256×256 (default)

Interpolation Method:

- INTER_AREA (optimized for downsampling)

This ensures consistency across dataset samples.

1.7 Quality Filtering

Each extracted clip undergoes validation to ensure dataset quality.

Checks Performed:

- Duration validation (valid clip length range)
- Motion validation (ensures active signing)

- Resolution check (minimum quality threshold)
- File integrity check (readable and non-corrupt)

Only clips passing all checks are included in the final dataset.

1.8 Statistics Generation

Dataset-level metadata is generated for analysis and reporting.

Metrics Include:

- Total number of clips
- Total duration
- Average clip size
- Resolution statistics
- Detection method used
- Quality rate

Output is stored in a structured JSON file (statistics.json).

1.9 Preview Generation

Visual previews are created to validate detection accuracy and dataset quality.

Types of Previews:

- Bounding box overlay visualization
- Frame-level detection samples
- Grid-based dataset overview

These help in manual inspection of detection performance.

Phase 2: Temporal Segmentation

This phase focuses on dividing the extracted sign language video into **fixed-duration clips** for model training. The main objective is to capture meaningful sign segments while preserving important temporal information such as preparation, stroke, and retraction phases.

A default **5-second clip duration** is used, which aligns with the specification for continuous sign language recognition. The system also supports **configurable clip durations**, allowing shorter windows for isolated signs and longer windows when needed.

An **overlap strategy (50%)** is applied between consecutive clips to preserve temporal continuity and reduce boundary loss between sign segments.

Although the segmentation process supports multiple clip lengths, the **raw 10–15 second buffer extraction** recommended in the specification is not implemented, limiting the ability to preserve broader contextual information before final segmentation.

Phase 3: Handling Synchronization and Decalage

This phase focuses on handling **interpreter lag (decalage)**, which is the delay between spoken audio and the corresponding signed interpretation. In broadcast sign language, the interpreter typically signs **after** the speech, so direct timestamp matching can create incorrect audio-sign alignment.

To address this, a **temporal offset** should be applied so that the sign video is extracted slightly later than the spoken audio. Based on the specification, an average **3.36-second lag offset** is recommended.

In addition, **buffer windows** should be added to preserve complete sign movements:

- **2 seconds pre-buffer** before the spoken segment
- **5 seconds post-buffer** after the spoken segment

Example:

If the audio segment is from **10s–15s**:

- With **3.36s offset**, the matching sign segment becomes **13.36s–18.36s**
- After adding buffers, the final extracted window becomes **8s–23.36s**

This ensures that the full sign, including preparation and completion phases, is captured without truncation. Currently, this lag compensation strategy is recognized in documentation but **not implemented**, which may lead to speech-sign misalignment in the dataset.

Phase 4: Synchronous Audio Extraction

This phase focuses on extracting the **audio segment simultaneously** with each video clip to preserve the temporal relationship between speech and sign language.

The extracted audio follows the required standard format: **mono channel, 16kHz sampling rate, and 16-bit PCM (.wav)**, which is suitable for automatic speech recognition models such as **Whisper**.

The extraction process is performed during clip generation, ensuring that each video segment has a corresponding audio segment with matching timestamps. This maintains synchronization between both modalities and supports accurate multimodal dataset creation.

This phase fully satisfies the specification requirements with **exact parameter matching and synchronous implementation**.

Phase 5: Skeletal Feature Extraction

This phase focuses on converting the extracted sign language video into **skeletal landmark data** to reduce background complexity and represent sign movements mathematically.

The specification recommends using **MediaPipe Holistic** to extract a reduced set of **85 key landmarks**, including hand, upper body, and facial points, followed by **uniform frame sampling (K=30)** to maintain temporal consistency.

In the current implementation, **MediaPipe Pose** is used only for interpreter detection and region extraction, not for feature extraction. The detected landmarks are used temporarily for bounding box generation and are not saved as output.

As a result, the system currently produces **RGB video clips instead of skeletal data**, meaning the intended benefits of background invariance, reduced data size, and motion-focused representation are not fully achieved.

Phase 6: Deferred Post-Processing (Transcription & Alignment)

This phase focuses on post-processing the extracted audio clips to generate **text transcriptions and align them with sign language data**. It is designed to run asynchronously after the dataset clips are created.

The system uses a **Whisper-based ASR model** to transcribe .wav audio files, with support for **Sinhala language (si)** and **word-level timestamps**. This allows precise mapping between spoken words and their temporal positions in the audio stream. Confidence scores and detailed word timing metadata are also stored for each clip.

However, two key specification components are not implemented:

- **SVO → SOV grammatical reordering**, which is required to adapt spoken Sinhala sentence structure into sign language (SSL) structure.
- **CTC-based weakly supervised alignment**, which is intended to learn sign boundaries without requiring frame-level annotation.

As a result, the current system successfully produces high-quality transcription and timestamp metadata, but does not yet perform linguistic restructuring or learned alignment between speech and sign sequences.